

FreeBSD Kernel Module for Networking

I want to create a FreeBSD kernel module to test networking, if I can send out arbitrary packets over an existing interface. To do this I am ☐ using [FreeBSD 13 Installation on a Laptop](#) which is 32-bit. The reason I use FreeBSD 13.5, is that I want to see if I can port this driver to [QNX 8](#).

1 FreeBSD Driver Documentation

The main page [Chapter 9. Writing FreeBSD Device Drivers](#) (<https://docs.freebsd.org/en/books/arch-handbook/driverbasics/>) isn't very helpful at all with:

Drivers for network devices do not use device nodes in order to be accessed. Their selection is based on other decisions made inside the kernel and instead of calling `open()`, use of a network device is generally introduced by using the system call `socket(2)`.

For more information see `ifnet(9)`, the source of the loopback device.

We can read the loopback device which is at `if_loop.c` (https://github.com/freebsd/freebsd-src/blob/release/13.5.0/sys/net/if_loop.c), which is reasonably easy to read (but wouldn't be easy to write for a beginner).

1.1 Getting the Sources

When installing to the SDCard for FreeBSD, it doesn't provide sources. Or, we might just not have installed the package. We need to get this manually. Update the branch based on the kernel sources you're using. In this case, I'm targetting FreeBSD 13.5 for network development. Instructions for more detailed information is on the [FreeBSD Handbook](#) (<https://docs.freebsd.org/en/books/handbook/cutting-edge/>)

```
# rmdir /usr/src
# git clone --branch releng/13.5 https://git.freebsd.org/src.git /usr/src
```

1.2 Building the Kernel

See [Chapter 10. Configuring the FreeBSD Kernel](#) (<https://docs.freebsd.org/en/books/handbook/kernelconfig/>), that talks about configuring and building the kernel. A reason why one would build the kernel is to test and understand code in the kernel, testing often.

1.3 Manual Pages

Interesting man pages are:

- [queue\(3\)](https://man.freebsd.org/cgi/man.cgi?query=queue&apropos=0&sektion=3&manpath=FreeBSD+13.5-RELEASE) (<https://man.freebsd.org/cgi/man.cgi?query=queue&apropos=0&sektion=3&manpath=FreeBSD+13.5-RELEASE>) for iterating over the queues in the data structure
- [ifnet\(9\)](https://man.freebsd.org/cgi/man.cgi?query=ifnet&apropos=0&sektion=9&manpath=FreeBSD+13.5-RELEASE) (<https://man.freebsd.org/cgi/man.cgi?query=ifnet&apropos=0&sektion=9&manpath=FreeBSD+13.5-RELEASE>)
- [iflib\(9\)](https://man.freebsd.org/cgi/man.cgi?query=iflib&sektion=9&apropos=0&manpath=FreeBSD+13.5-RELEASE) (<https://man.freebsd.org/cgi/man.cgi?query=iflib&sektion=9&apropos=0&manpath=FreeBSD+13.5-RELEASE>) Network Interface
- [mbuf\(9\)](https://man.freebsd.org/cgi/man.cgi?query=mbuf&sektion=9&apropos=0&manpath=FreeBSD+13.5-RELEASE) (<https://man.freebsd.org/cgi/man.cgi?query=mbuf&sektion=9&apropos=0&manpath=FreeBSD+13.5-RELEASE>)

1.3.1 Installing the Manual Pages

You must have the sources `/usr/src` installed.

```
# cd /usr/src
# make -C share/man/man9
# make -C share/man/man9 install
```

Then you can view the man pages, such as:

```
$ man 9 malloc
```

2 FreeBSD Source Code for Kernel

The source code can be obtained by [GitHub](#) (<https://github.com/freebsd/freebsd-src/tree/main>). The [Release Engineering](#) (<https://www.freebsd.org>)

[g/releng/](#) page contains information about the branches used. For example, the motivation for this is QNX8 development. Looking at QNX8 io-sock, we see:

```
main_buffer* 1 io-sock -m phy -m fdt -m phy_fdt -d genet -m pci -d em -d ix -d re -d igc -m usb -d axe -d axge -d cdce -d smsc
main_buffer 1 io-sock fullpath: /sbin/io-sock
main_buffer 1 io-sock BuildID 118221426d88c69d642130807c363bb4
main_buffer 1 Copyright (c) 1992-2021 The FreeBSD Project.
main_buffer 1 Copyright (c) 1979, 1980, 1983, 1986, 1988, 1989, 1991, 1992, 1993, 1994
main_buffer 1 The Regents of the University of California. All rights reserved.
main_buffer 1 FreeBSD is a registered trademark of The FreeBSD Foundation.
main_buffer 1 FreeBSD 13.5-RELEASE-p1 #0 n2105-925fe7e1e: Wed Jul 30 19:04:00 EDT 2025
main_buffer 1 root@docker-n5:/builds/workspace/QNX05_803/build_aarch64/lib/io-sock/sys/target/aarch64-o.le
```

i.e. we would like to look at the code from FreeBSD 13.5-RELEASE-p1

- The branch stable/13 is for the development of 13.x
- The branch releng/13.5 is for security fixes of 13.5
- There is a tag release/13.5.0-p1 on the branch releng/13.5 that probably contains what we want. I couldn't find a commit 925fe7e1e.

```
error: pathspec '925fe7e1e' did not match any file(s) known to git
```

When downloading from FreeBSD (<https://download.freebsd.org/releases/amd64/13.5-RELEASE/>), there is a REVISION (<https://download.freebsd.org/releases/amd64/13.5-RELEASE/REVISION>) file that gives the GIT hash. At this time, this is the same hash as the tag release/13.5.0, so earlier than the security patches.

3 A Very Simple Network Driver

I found some code (<https://forums.freebsd.org/threads/how-to-specify-kernel-configuration-when-compiling-out-of-tree-kernel-modules.71576/>) that compiles and runs on FreeBSD 13.5. Note, analysis below shows that this code should not be used (it has subtle race conditions).

Code: MakeFile

```
KMOD=    showifn
SRCS=    showifn.c
.include <bsd.kmod.mk>
```

Code: showifn.c

```
#include <sys/param.h>
#include <sys/module.h>
#include <sys/kernel.h>
#include <sys/systm.h>

#include <sys/types.h>
#include <sys/socket.h>
#include <net/if.h>
#include <net/if_var.h>

static void
show_ifn(void)
{
    struct ifnet *ifp;

    IFNET_RLOCK();
    CK_STAILQ_FOREACH(ifp, &V_ifnet, if_link) {
        printf(" %s via %s\n", ifp->if_xname, ifp->if_dname);
    }
    IFNET_RUNLOCK();
}

static int
kmod_main(struct module *module, int event, void *arg)
{
    int res = 0;

    switch (event) {
        case MOD_LOAD:
            printf("showifn loading\n");
            show_ifn();
            break;

        case MOD_UNLOAD:
            printf("showifn unloading\n");
            break;

        default:
            res = EOPNOTSUPP;
            break;
    }

    return res;
}

static moduledata_t showifn_conf = {
    "showifn",
    kmod_main,
    NULL
}
```

```
};

DECLARE_MODULE(showifn, showifn_conf, SI_SUB_DRIVERS, SI_ORDER_MIDDLE);
```

To build:

```
$ make
machine -> /usr/src/sys/i386/include
x86 -> /usr/src/sys/x86/include
touch opt_global.h
Warning: Object directory not changed from original /usr/home/jcurl/src
cc -O2 -pipe -fno-strict-aliasing -Werror -D_KERNEL -DKLD_MODULE -nostdinc -include /usr/home/jcurl/src/opt_global.h -I. -I/usr/src/sys -I/usr/src/sys/contrib/ck/include -fno-common -fdebug-prefix-map=/machine=/usr/src/sys/i386/include -fdebug-prefix-map=/x86=/usr/src/sys/x86/include -MD -MF.depend.showifn.o -MTshowifn.o -mno-mmx -mno-sse -msoft-float -ffreestanding -fwrapv -fstack-protector -Wall -Wstrict-prototypes -Wmissing-prototypes -Wpointer-arith -Wcast-qual -Wundef -Wno-pointer-sign -D__printf__=__freebsd_kprintf__ -Wmissing-include-dirs -fdiagnostics-show-option -Wno-unknown-pragmas -Wno-error=tautological-compare -Wno-error=empty-body -Wno-error=parentheses-equality -Wno-error=unused-function -Wno-error=pointer-sign -Wno-error=shift-negative-value -Wno-error=address-of-packed-member -Wno-error=array-parameter -Wno-error=deprecated-non-prototype -Wno-error=strict-prototypes -Wno-error=unused-but-set-variable -Wno-error=unused-but-set-variable -Wno-format-zero-length -mno-aes -mno-avx -std=iso9899:1999 -c showifn.c -o showifn.o
ld -m elf_i386_fbsd -warn-common --build-id=shal -T /usr/src/sys/conf/ldscript.kmod.i386 -r -o showifn.kld showifn.o
-> export_syms
awk -f /usr/src/sys/conf/kmod_syms.awk showifn.kld export_syms | xargs -J% objcopy % showifn.kld
ld -m elf_i386_fbsd -Bshareable -znotext -znorelro -warn-common --build-id=shal -o showifn.ko showifn.kld
objcopy --strip-debug showifn.ko
```

To run the driver:

```
$ sudo kldload ./showifn.ko
```

To unload the driver

```
$ sudo kldunload ./showifn.ko
```

To view the output:

```
$ sudo dmesg
showifn loading
em0 via em
lo0 via lo
showifn unloading
```

4 FreeBSD APIs

Research is done on FreeBSD 13.5.

4.1 Memory Allocation

See [malloc\(9\)](https://man.freebsd.org/cgi/man.cgi?query=malloc&sektion=9&manpath=FreeBSD+13.5-RELEASE) (<https://man.freebsd.org/cgi/man.cgi?query=malloc&sektion=9&manpath=FreeBSD+13.5-RELEASE>)

Kernel modules then do:

- `MALLOC_DECLARE(M_BPF);` declares the memory type `M_BPF`
- `MALLOC_DEFINE(M_BPF, "BPF", "BPF data");` defines the memory type `M_BPF`
- `bp = malloc(sizeof(*bp), M_BPF, M_WAITOK | M_ZERO);` allocates the type `M_BPF`
 - Note, when using `M_WAITOK`, the `malloc()` can never return `NULL`.
- `free(bp, M_BPF);` Frees the memory

Notice that all memory is tagged. You can see the memory usage in the kernel with:

```
$ vmstat -m
```

e.g.

```
$ vmstat -m | grep -i bpf
Type InUse MemUse Requests Size(s)
BPF 10 18K 10 128,256,512,4096
```

4.2 Queues

FreeBSD uses queues and lists for the datastructures. The `CK_*` is defined by the concurrency kit (https://man.freebsd.org/cgi/man.cgi?query=ck_queue&sektion=3&manpath=FreeBSD+13.5-RELEASE+and+Ports), which allows lockless access. See [QUEUE\(3\)](https://man.freebsd.org/cgi/man.cgi?query=STAILQ_FOREACH&manpath=FreeBSD+13.5-RELEASE+and+Ports) (https://man.freebsd.org/cgi/man.cgi?query=STAILQ_FOREACH&manpath=FreeBSD+13.5-RELEASE+and+Ports) for information about using the queues.

- `CK_STAILQ_HEAD(HEADNAME, TYPE);` . A singly-linked tail queue is headed by a structure defined by the `STAILQ_HEAD` macro. This structure contains a pair of pointers, one to the first element in the tail queue and the other to the last element in the tail queue. The elements are singly linked for minimum space and pointer manipulation overhead at the expense of $O(n)$ removal for arbitrary elements. New elements can be added to the tail queue after an existing element, at the head of the tail queue, or at the end of the tail queue. A `STAILQ_HEAD` structure is declared as follows:

```
STAILQ_HEAD(HEADNAME, TYPE) head;
```

where `HEADNAME` is the name of the structure to be defined, and `TYPE` is the type of the elements to be linked into the tail queue. A pointer to the head of the tail queue can later be declared as:

```
struct HEADNAME *headp;
```

(The names `head` and `headp` are user selectable.)

- `CK_STAILQ_ENTRY(TYPE);` . In the macro definitions, `TYPE` is the name of a user defined structure. The structure must contain a field called `NAME` which is of type `SLIST_ENTRY`, `STAILQ_ENTRY`, `LIST_ENTRY`, or `TAILQ_ENTRY`. The macro `STAILQ_ENTRY` declares a structure that connects the elements in the tail queue.
- `CK_STAILQ_FOREACH(TYPE *var, CK_STAILQ_HEAD *head, CK_STAILQ_ENTRY NAME);` . The macro `STAILQ_FOREACH` traverses the tail queue referenced by `head` in the forward direction, assigning each element in turn to `var`.

4.3 Network Interface List

Documentation for network interfaces is primarily at `ifnet(9)` (<https://man.freebsd.org/cgi/man.cgi?query=ifnet&sektion=9&manpath=FreeBSD+13.5-RELEASE>). Documentation of how network packets are queued are described by `altq(9)` (<https://man.freebsd.org/cgi/man.cgi?query=altq&sektion=9&manpath=FreeBSD+13.5-RELEASE>).

Some important datatypes defined in `if_var.h`:

- `typedef struct ifnet * if_t;`

4.3.1 Lock

Defined in `if.c`:

```
/*
 * The global network interface list (V_ifnet) and related state (such as
 * if_index, if_indexlim, and ifindex_table) are protected by an sxlock.
 * This may be acquired to stabilise the list, or we may rely on NET_EPOCH.
 */
struct sx ifnet_sxlock;
SX_SYSINIT_FLAGS(ifnet_sx, &ifnet_sxlock, "ifnet_sx", SX_RECURSE);
```

- `IFNET_RLOCK();` : Defined in `if_var.h` and is `sx_slock` (https://man.freebsd.org/cgi/man.cgi?query=sx_slock&manpath=FreeBSD+13.5-RELEASE) (`&ifnet_sxlock`).
- `IFNET_RUNLOCK();` : Defined in `if_var.h` and is `sx_sunlock` (https://man.freebsd.org/cgi/man.cgi?query=sx_sunlock&manpath=FreeBSD+13.5-RELEASE) (`&ifnet_sxlock`).

4.3.2 The list of interfaces (ifnethead)

At the top of `if_var.h`:

```
CK_STAILQ_HEAD(ifnethead, ifnet); /* we use TAILQs so that the order of */
CK_STAILQ_HEAD(ifaddrhead, ifaddr); /* instantiation is preserved in the list */
CK_STAILQ_HEAD(ifmultihead, ifmultiaddr);
CK_STAILQ_HEAD(ifgrouphead, ifg_group);
```

The `CK_STAILQ_HEAD` is defined as:

```
#define CK_STAILQ_HEAD(name, type) \
struct name { \
    struct type *cstqh_first; /* first element */ \
    struct type **cstqh_last; /* addr of last next element */ \
}
```

So that this code defines:

```
struct ifnethead {
```

```

    struct ifnet *cstqh_first;
    struct ifnet **cstqh_last;
}

```

4.3.3 VNET (virtualised network stacks)

Later in `if_var.h`:

```

VNET_DECLARE(struct ifnethead, ifnet);

#define V_ifnet VNET(ifnet)

```

which in the simplest case is the same as `(vnet.h)`:

```

extern struct ifnethead ifnet
#define V_ifnet ifnet

```

The datatype `struct ifnethead` is defined earlier by `CK_STAILQ_HEAD(ifnethead, ifnet)`

4.3.4 Iterating Interfaces

So the code:

```

struct ifnet *ifp;

CK_STAILQ_FOREACH(ifp, &V_ifnet, if_link) {
    printf(" %s via %s\n", ifp->if_xname, ifp->if_dname);
}

```

takes the `(&V_ifnet)` as the head, `if_link` as the field.

We see that `struct ifnet` is a large datastructure defined in `if_var.h`. It forms a linked list, allowing iterating to the next entry with the field:

```

struct ifnet {
    CK_STAILQ_ENTRY(ifnet) if_link; /* all struct ifnets are chained (CK_) */
}

```

So, we can see it is iterating from the first element `struct ifnet *ifp = ifnet->cstqh_first`; and getting the next entry via `ifp = ifp->if_link`.

4.3.5 Finding an Interface

One shouldn't try and iterate the interfaces yourself. The `struct ifnet` has reference counting that helps maintain the lifetime of the reference.

There is the function `ifunit` and `ifunit_ref` both defined in `ifnet(9)` (<https://man.freebsd.org/cgi/man.cgi?query=ifunit&manpath=FreeBSD+13.5-RELEASE>).

`ifunit()`: Return an ifnet pointer for the interface named name.

`ifunit_ref()`: Return a reference-counted (via `ifa_ref()`) ifnet pointer for the interface named name. This is the preferred function over `ifunit()`. The caller is responsible for releasing the reference with `if_rele()` when it is finished with the ifnet.

The code is simple:

Code: `sys/net/if.c`

```

struct ifnet *
ifunit_ref(const char *name)
{
    struct epoch_tracker et;
    struct ifnet *ifp;

    NET_EPOCH_ENTER(et);
    CK_STAILQ_FOREACH(ifp, &V_ifnet, if_link) {
        if (strcmp(name, ifp->if_xname, IFNAMSIZ) == 0 &&
            !(ifp->if_flags & IFF_DYING))
            break;
    }
    if (ifp != NULL) {
        if_ref(ifp);
    }

    NET_EPOCH_EXIT(et);
    return (ifp);
}

```

```

struct ifnet *
ifunit(const char *name)
{
    struct epoch_tracker et;
    struct ifnet *ifp;

    NET_EPOCH_ENTER(et);
    CK_STAILQ_FOREACH(ifp, &V_ifnet, if_link) {
        if (strcmp(name, ifp->if_xname, IFNAMSIZ) == 0)
            break;
    }
    NET_EPOCH_EXIT(et);
    return (ifp);
}

```

4.3.6 Epochs in the Kernel

Refer to `epoch(9)` (<https://man.freebsd.org/cgi/man.cgi?query=epoch&manpath=FreeBSD+13.5-RELEASE>). The `NET_EPOCH_ENTER` is defined at `sys/epoch.h` as (with the trace code stripped, just to make it simpler to read):

Code: `sys/sys/epoch.h`

```

<syntaxhighlight lang="c">
void _epoch_enter_preempt(epoch_t epoch, epoch_tracker_t et);
void _epoch_exit_preempt(epoch_t epoch, epoch_tracker_t et);
#define epoch_enter_preempt(epoch, et) _epoch_enter_preempt(epoch, et)
#define epoch_exit_preempt(epoch, et) _epoch_exit_preempt(epoch, et)
void epoch_enter(epoch_t epoch);
void epoch_exit(epoch_t epoch);

/*
 * Globally recognized epochs in the FreeBSD kernel.
 */
/* Network preemptible epoch, declared in sys/net/if.c. */
extern epoch_t net_epoch_preempt;
#define NET_EPOCH_ENTER(et) epoch_enter_preempt(net_epoch_preempt, &(et))
#define NET_EPOCH_EXIT(et) epoch_exit_preempt(net_epoch_preempt, &(et))
#define NET_EPOCH_WAIT() epoch_wait_preempt(net_epoch_preempt)
#define NET_EPOCH_CALL(f, c) epoch_call(net_epoch_preempt, (f), (c))
#define NET_EPOCH_DRAIN_CALLBACKS() epoch_drain_callbacks(net_epoch_preempt)
#define NET_EPOCH_ASSERT() MPASS(in_epoch(net_epoch_preempt))

```

From the Kernel pages:

- `epoch_enter_preempt`: The start of a critical section for the current kernel thread. Only non-sleepable locks can be acquired during the critical section.
- `epoch_exit_preempt`: Marks the end of the critical section.

The critical section is defined by `extern epoch_t net_epoch_preempt;`.

When using network code, it is important to use `NET_EPOCH_ENTER` and `NET_EPOCH_EXIT`, as other code can, asynchronously, call `NET_EPOCH_CALL` that will execute a function asynchronously when the call is free.

4.3.7 Fields of the ifnet

- `if_dname`: The driver name (e.g. genet or em)
- `if_xname`: The external name (e.g. em0)
- `if_mtu`: The configured MTU
- `if_addrln` and `if_hw_addr`: The MAC address
 - Need to check that it's of the ETHERNET type with `if_type`
- `if_snd`: The output queue, of type `struct ifaltq` and covered by `altq(9)` (<https://man.freebsd.org/cgi/man.cgi?query=altq&sektion=9&manpath=FreeBSD+13.5-RELEASE>).

4.3.8 Detecting Interfaces Up or Down

The `EVENTHANDLER` (<https://man.freebsd.org/cgi/man.cgi?query=EVENTHANDLER&sektion=9&manpath=FreeBSD+13.5-RELEASE>) interface can be used to register with the FreeBSD kernel to identify when an interface link state changes.

In particular, we start by analysing the code `if_detach()` in `if.c`. It calls `if_detach_internal()` which in turn calls `if_down()`. The code is:

Code: `sys/net/if.c`

```

/*
 * Mark an interface down and notify protocols of
 * the transition.
 */
void
if_down(struct ifnet *ifp)
{

```

```

    EVENTHANDLER_INVOKE(ifnet_event, ifp, IFNET_EVENT_DOWN);
    if_unroute(ifp, IFF_UP, AF_UNSPEC);
}

```

This suggests we can register for the event `ifnet_event` to get information. In our module load/unload:

Code: showifn.c

```

static void
showifn_link_event(void *arg, struct ifnet *ifp, int linkstate)
{
    if (ifp == NULL) return;

    if (linkstate == LINK_STATE_UP) {
        mod_printf("interface %s up\n", ifp->if_xname);
    } else if (linkstate == LINK_STATE_DOWN) {
        mod_printf("interface %s is down\n", ifp->if_xname);
    } else {
        mod_printf("interface %s has linkstate %d\n", ifp->if_xname, linkstate);
    }
}

static void
showifn_event(void *arg, struct ifnet *ifp, int event)
{
    if (ifp == NULL) return;

    if (event == IFNET_EVENT_DOWN) {
        mod_printf("interface %s down\n", ifp->if_xname);
    } else if (event == IFNET_EVENT_UP) {
        mod_printf("interface %s up\n", ifp->if_xname);
    } else {
        mod_printf("interface %s event %d\n", ifp->if_xname, event);
    }
}

static int
kmod_main(struct module *module, int event, void *arg)
{
    static eventhandler_tag link_event;
    static eventhandler_tag if_event;
    int res = 0;

    switch (event) {
        case MOD_LOAD:
            link_event = EVENTHANDLER_REGISTER(ifnet_link_event,
                                                showifn_link_event,
                                                NULL,
                                                EVENTHANDLER_PRI_ANY);
            if_event = EVENTHANDLER_REGISTER(ifnet_event,
                                              showifn_event,
                                              NULL,
                                              EVENTHANDLER_PRI_ANY);

            show_ifn();
            break;

        case MOD_UNLOAD:
            EVENTHANDLER_DEREGISTER(ifnet_link_event, link_event);
            EVENTHANDLER_DEREGISTER(ifnet_event, if_event);
            break;

        default:
            res = EOPNOTSUPP;
            break;
    }

    return res;
}

```

When testing with:

```

# ifconfig lo0 down
# dmesg
lo0: link state changed to DOWN
showifn: interface lo0 link down

```

and similarly:

```

# ifconfig lo0 up
# dmesg
lo0: link state changed to UP
showifn: interface lo0 link up
showifn: interface lo0 up

```

For VLAN interfaces, we get status the interface is up (but not the link)

```

# ifconfig vlan73 create vlan 73 vlandev genet0
# dmesg
showifn: interface vlan73 up
# ifconfig
vlan73: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500
options=80000<LINKSTATE>
ether d8:3a:dd:a5:3d:8a
inet6 fe80::da3a:ddff:fea5:3d8a%vlan73 prefixlen 64 scopeid 0x3
groups: vlan
vlan: 73 vlanproto: 802.1q vlanpcp: 0 parent interface: genet0

```

```
media: Ethernet autoselect (1000baseT <full-duplex>)  
status: active  
nd6 options=23<PERFORMNUD,ACCEPT_RTADV,AUTO_LINKLOCAL>
```

Then there are no events for the following, and even the `ifconfig` shows the interface is still **RUNNING**.

```
# ifconfig vlan73 down  
# dmesg  
showifn: interface vlan73 down  
-----  
# ifconfig vlan73 up  
# dmesg  
showifn: interface vlan73 up  
-----  
# ifconfig vlan73 destroy  
# dmesg  
showifn: interface vlan73 down
```

Other useful events might be:

- `EVENTHANDLER_INVOKE(ifnet_link_event, ifp, link_state);`
- `EVENTHANDLER_INVOKE(ifnet_arrival_event, ifp);`
- `EVENTHANDLER_INVOKE(ifnet_departure_event, ifp);`
- `EVENTHANDLER_INVOKE(group_attach_event, ifg);`
- `EVENTHANDLER_INVOKE(group_change_event, groupname);`
- `EVENTHANDLER_INVOKE(ifnet_event, ifp, IFNET_EVENT_DOWN);`
- `EVENTHANDLER_INVOKE(ifnet_event, ifp, IFNET_EVENT_UP);`
- `EVENTHANDLER_INVOKE(iflladdr_event, ifp);`

Retrieved from "https://www.home.lan/wiki/index.php?title=FreeBSD_Kernel_Module_for_Networking&oldid=12805"

This page was last edited on 23 October 2025, at 22:50.